CareerOrbit V3.1 — Backend & AI Architecture Bible

> **Audience:** Backend Engineers, AI Architects, Data Scientists **Objective:** A comprehensive, audited technical manual for the "Invisible Brain" of CareerOrbit.

---

1. System Vision & Serverless Architecture

CareerOrbit operates on a **Serverless-First** paradigm, utilizing Next.js 14 edge and lambda functions to deliver a deterministic, adaptive learning experience.

Technical Stack

- **Runtime:** Next.js 14 (App Router).
- **Database:** Neon PostgreSQL (Serverless) with connection pooling.
- **ORM:** Drizzle ORM (Type-safe SQL schema).
- **Auth:** NextAuth.js (JWT Strategy).
- **AI Core:** Vercel AI SDK with OpenAI provider.

---

2. Database Layer: Cognitive Data Structures

We use a hybrid relational/document model, leveraging PostgreSQL **JSONB** for high-variance cognitive data.

A. The `user_model` Table (The Memory Center)

This table stores the unified cognitive snapshot for every learner.

- `knowledgeState` (JSONB): A dictionary of Knowledge Component (KC) states.
  - Structure: `{ [subtopicId: string]: { pMastery: number, attempts: number, correctCount: number, lastUpdated: string } }`
- `capabilityScore`: Computed as `mean(pMastery) * 100`.
- `learningVelocity`: Tracks the rate of cognitive gain ('low', 'medium', 'high').
- `consistencyScore`: Measures the temporal regularity of learning events.

B. The `roadmaps` Table

- `modules` (JSONB): A nested array of modules containing subtopics, practical tasks, and **NOS Codes**.
- `calibrationStatus`: Tracks if the roadmap has been fully adapted to the user's BKT baseline.

C. The `learning_events` Table (Research Audit Trail)

An immutable log of every cognitive shift. Essential for calculating **Hake Gain** and exporting research data.

- `eventType`: 'quiz_answer', 'bkt_update', 'module_complete', 'pre_test'.

---

3. The Mastery Engine: BKT-IRT Hybrid Specification

Our engine implements a modified **Bayesian Knowledge Tracing** model enhanced with **Item Response Theory (IRT)** heuristics.

Base Parameters (`DEFAULT_BKT_PARAMS`)

- `pL0` (Initial Mastery): **0.10**
- `pT` (Learning Rate): **0.20**
- `pG` (Base Guess): **0.25**
- `pS` (Base Slip): **0.10**

IRT Dynamic Parameter Scaling

The engine scales pG and pS based on the question difficulty provided by the AI:

- **Easy:** pG scales up to **0.40**; pS drops to **0.05**.
- **Hard:** pG drops to **0.10**; pS scales up to **0.25**.

Breakthrough Recovery Logic

To mitigate the "Zero Trap," a **Recovery Boost** is applied if a user with $pMastery < 0.20$ provides a correct answer. The pG penalty is halved ($pG \times 0.5$), assuming learning rather than a lucky guess.

Asymptotic Clamping

To maintain Markov chain responsiveness, $pMastery$ is strictly bounded to the interval $[0.05, 0.95]$. This prevents the model from saturating at 0 or 1.

---

4. AI Engineering & Prompt Registry

We treat the LLM as a structured content generator, strictly enforced via **Zod Schemas**.

Model Selection Strategy

- **Roadmap Generation:** Uses `gpt-5.4` via `getGPT5ReasoningModel()` for deep architectural reasoning and NSQF alignment.
- **Mentor & Instant Tasks:** Uses `gpt-4o-mini` via `getGPT5InstantModel()` for high-speed, low-latency empathetic guidance.

Prompt Protocols

1. **NSQF Enforcement:** All generative prompts (Roadmap/Recalibrate) enforce the 5 official NSQF domains and realistic NOS codes.
2. **BKT Injection:** The `buildBKTRecalibrationSummary()` function converts raw JSON into a human-readable "Knowledge State Analysis" for the LLM, enabling it to make informed scaffolding decisions.
3. **Scaffolding Levels:**
    - **Maximum (< 0.30):** Tiny steps, high scaffolding.
    - **Moderate (0.30 - 0.60):** Guiding questions.
    - **Minimal (0.60 - 0.85):** Independent challenge.
    - **None (> 0.85):** Mastery confirmed.

---

5. Analytics & Research Metrics

We utilize the **Normalized Learning Gain (NLG)** metric (Hake, 1998) to evaluate system efficacy.

Formula:

$$NLG = \frac{CurrentAvgMastery - BaselineAvgMastery}{1.0 - BaselineAvgMastery} \times 100$$

This controls for the "Ceiling Effect," allowing us to accurately compare learning velocity across different user demographics.

---

6. Performance & Scale Guidelines

- **Connection Pooling:** Use the `pool` instance for all Neon DB transactions to manage serverless connection overhead.
- **Revalidation:** Use `revalidatePath('/learn')` or tag-based revalidation after every BKT update to ensure the UI reflects the new cognitive state instantly.
- **Structured IO:** Never use raw string parsing. Every LLM call must utilize `streamObject` or `generateObject` with a validated Zod schema.